JWT Security Analysis Report

Prepared by: *Annie Stephen Bassey*
Date: *26/11/2025*
Target Component: JSON Web Token (Authentication Layer)
Testing Window: *Single-token security assessment*
Testing Type: Security Analysis

1. Executive Summary

This report documents a security analysis performed on a provided JWT.
The objective was to evaluate the token's structure, configuration, and security controls to identify vulnerabilities that could lead to authentication bypass, privilege escalation, or data exposure.

The analysis revealed **five critical weaknesses**, primarily stemming from the use of:

- The none algorithm (no signature)
- Client-controlled authorization data (role & validity)
- Expired or unvalidated timestamps
- Plaintext, non-encrypted payload
- Trusting integrity of client-side data

2. Scope

In Scope

- JWT provided:

`eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyIjoiYWRtaW4iLCJleHAiO`

jE2MDAwMDAwMDAsInJvbGUiOiJhZG1pbiIsInZhbGlkIjpmYWxzZX0.

- Header check
- Payload integrity check
- Signature validation
- Token security configuration

3. Methodology

The following process was followed to assess the token:

Tools Used

- **jwt.io** (decode & inspect)
- **Manual Base64 examination**
- **Standard JWT security controls review**

Process Overview

1. Decode header and payload
2. Review signature and algorithm configuration
3. Assess token integrity & tamper resistance
4. Validate timestamps (e.g., expiration)
5. Examine exposure of sensitive data
6. Identify access-control or authentication weaknesses

4. Key Findings

Finding 1: Use of (`alg: none`) No Signature Applied

Description:

The token header carries a `"alg":"none",` which disables cryptographic signing entirely. This means the server accepts the token without any form of verification of its integrity or origin.

How attackers can use it :

Attackers can modify any part of the token including user and role while maintaining a valid structure.

If accepted by the server, this allows:

- Full authentication bypass
- Privilege escalation
- user impersonation
- Complete compromise of protected APIs

Finding 2: Admin Role Stored in Plaintext in the Payload

Description:

The `"role": "admin"` field is included directly inside the client-side of payload, meaning access level decisions may depend on values the attacker can freely edit.

How attackers can use it:

If the server trusts the role value:

- Any attacker could elevate privileges simply by rewriting `"role":"admin"`
- Administrative functions could be accessed without authentication

Finding 3: Token Contains an Expired Value

Description:

The `"exp"` timestamp (`1600000000`) corresponds to September 2020.
This token is long expired.

If a server still accepts this token:

- Expiration validation is missing
- Replay attacks become possible
- Old, compromised tokens may remain indefinitely valid

How attackers can use it:

Attackers could reuse stale tokens to impersonate legitimate users or retain access long after revocation.

Finding 4: Presence of a Flag Suggests Client-Side Trust

Description:

The payload contains a field:

```
"valid": false
```

This should *never* be stored client-side, as it indicates the server might base authentication decisions on a value the attacker controls.

How attackers can use it:

An attacker could modify it to:

```
"valid": true
```

leading to:

- The attacker make acceptance of tokens from the client side
- Authentication bypass

Finding 5: Token Payload Not Encrypted ; Sensitive Claims Exposed

Description:

This bearer token is not encrypted and the signature algo carries a “none” value which means it is not signed
And exposes all sensitive fields in plaintext:

- Username
- Role
- Auth flags
- Internal logic indicators

How attackers can use it:

Attackers can:

- Recon system behavior
- Learn expected roles and token structure
- Craft forged admin-level tokens
- Identify weaknesses in access-control design

JWT Debugger

Debugger | Introduction | Libraries | Ask

ENCODED VALUE

Enable auto-focus

JSON WEB TOKEN (JWT)

COPY CLEAR

Valid JWT

This is an Unsecured JWT as defined by Section 6 of RFC 7519.

eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyIjoiYWRtaW4iLCJleHAiOjE2MDAwMDAwMDAsInJvbGUiOiJhZG1pbiIsInZhbGlkIjpmYWxzZX0.

DECODED HEADER

JSON CLAIMS TABLE COPY

```
{
  "alg": "none",
  "typ": "JWT"
}
```

DECODED PAYLOAD

JSON CLAIMS TABLE COPY

Sun Sep 13 2020 13:26:40 GMT+0100 (West Africa Standard Time)

```
{
  "user": "admin",
  "exp": 1600000000,
  "role": "admin",
  "valid": false
}
```

JWT Debugger

Debugger | Introduction | Libraries | Ask

JSON WEB TOKEN (JWT)

COPY CLEAR

Valid JWT

This is an Unsecured JWT as defined by Section 6 of RFC 7519.

eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJ1c2VyIjoiYWRtaW4iLCJleHAiOjE2MDAwMDAwMDAsInJvbGUiOiJhZG1pbiIsInZhbGlkIjpmYWxzZX0.

JSON CLAIMS TABLE HIDE DETAILS

alg	none	The algorithm used to sign the JWT. Learn more
typ	JWT	The media type of this complete JWT. Learn more

DECODED PAYLOAD

JSON CLAIMS TABLE HIDE DETAILS

user	admin	
exp	1600000000 (Sun Sep 13 2020 13:26:40 GMT+0100 (West Africa Standard Time))	The expiration time on or after which the JWT MUST NOT be accepted for processing. Learn more
This value must be a NumericDate type, representing seconds.		
role	admin	
valid	false	